

ТЕХНІЧНА ДОКУМЕНТАЦІЯ

MyOwnAI

Локальний AI-асистент з RAG-архітектурою —
від нуля до розуміння

React

FastAPI

Ollama

LangChain

Qdrant

Docker

Llama 3

Версія 1.0 · Травень 2026 · Для початківців у цьому стеку

Що є в цьому документі

1

Що таке RAG і навіщо це потрібно

Пояснення концепції Retrieval-Augmented Generation простими словами

2

Загальна архітектура системи

Усі 4 сервіси, як вони з'єднані, схема взаємодії

3

Сервіси та технології

Навіщо кожен компонент: Docker, Qdrant, Ollama, LangChain

4

Потік даних: як працює запит

Покроково від введення тексту до відповіді AI

5

Backend — FastAPI

API ендпоінти, структура коду, ключові функції

6

Frontend — React

Компоненти UI, стан додатку, взаємодія з API

7

Запуск та розгортання

Docker Compose, Makefile команди, перший запуск

8

Типові проблеми та рішення

FAQ, помилки та як їх виправляти

Що таке RAG і навіщо це потрібно

RAG (Retrieval-Augmented Generation) — це підхід, де AI-модель відповідає не лише з власних знань, а й використовує конкретні документи, які ти завантажив. Думай про це як про ChatGPT, якому ти можеш підсунути свої нотатки перед тим, як він відповідатиме.

Проблема звичайних AI-моделей

Велика мовна модель (LLM) знає тільки те, на чому її навчали. Вона не знає:

- Твоїх особистих документів і нотаток
- Внутрішньої документації твоєї компанії
- Подій після дати навчання моделі
- Специфічної інформації твого проєкту

// ПРОБЛЕМА: ЗВИЧАЙНА LLM БЕЗ RAG

Питання: "Що написано в нашому корпоративному регламенті?"

LLM (Llama 3, GPT тощо)

Знає: загальні знання зі свого навчання

НЕ знає: твої документи, корпоративні дані



Відповідь: "Вибачте, я не маю доступу до цього документу"

// РІШЕННЯ: RAG — ДОДАЄМО СВОЇ ДОКУМЕНТИ

Питання: "Що написано в нашому корпоративному регламенті?"



КРОК 1: Пошук (Retrieval)

Питання → перетворюємо у вектор

Шукаємо схожі вектори в базі документів

Знаходимо топ-3 найрелевантніших шматки тексту



[знайдений контекст]



КРОК 2: Генерація (Generation)

Надсилаємо до LLM:

"Ось контекст: [знайдені документи]"

"Відповідь на питання, спираючись на нього"



Відповідь: точна, з посиланням на конкретний документ ✓



Аналогія для розуміння

Уяви, що ти здаєш іспит з відкритою книгою. Без RAG — ти відповідаєш лише з пам'яті. З RAG — ти спочатку знаходиш потрібну сторінку в книзі, а потім формулюєш відповідь на основі неї. Результат точніший і прив'язаний до реального джерела.

Ключове поняття: Вектори (Embeddings)

Щоб комп'ютер міг "шукати схожі тексти", ми перетворюємо текст у числові вектори. Схожі за змістом тексти матимуть схожі вектори.

```
// ЯК ТЕКСТ ПЕРЕТВОРЮЄТЬСЯ У ВЕКТОР
```

"Кішка сидить на даху"

|

▼ `openai-embed-text` (768-вимірний вектор)

|

`[0.23, -0.87, 0.41, 0.02, ..., 0.95]` ← 768 чисел

|

Відстань між векторами = семантична схожість

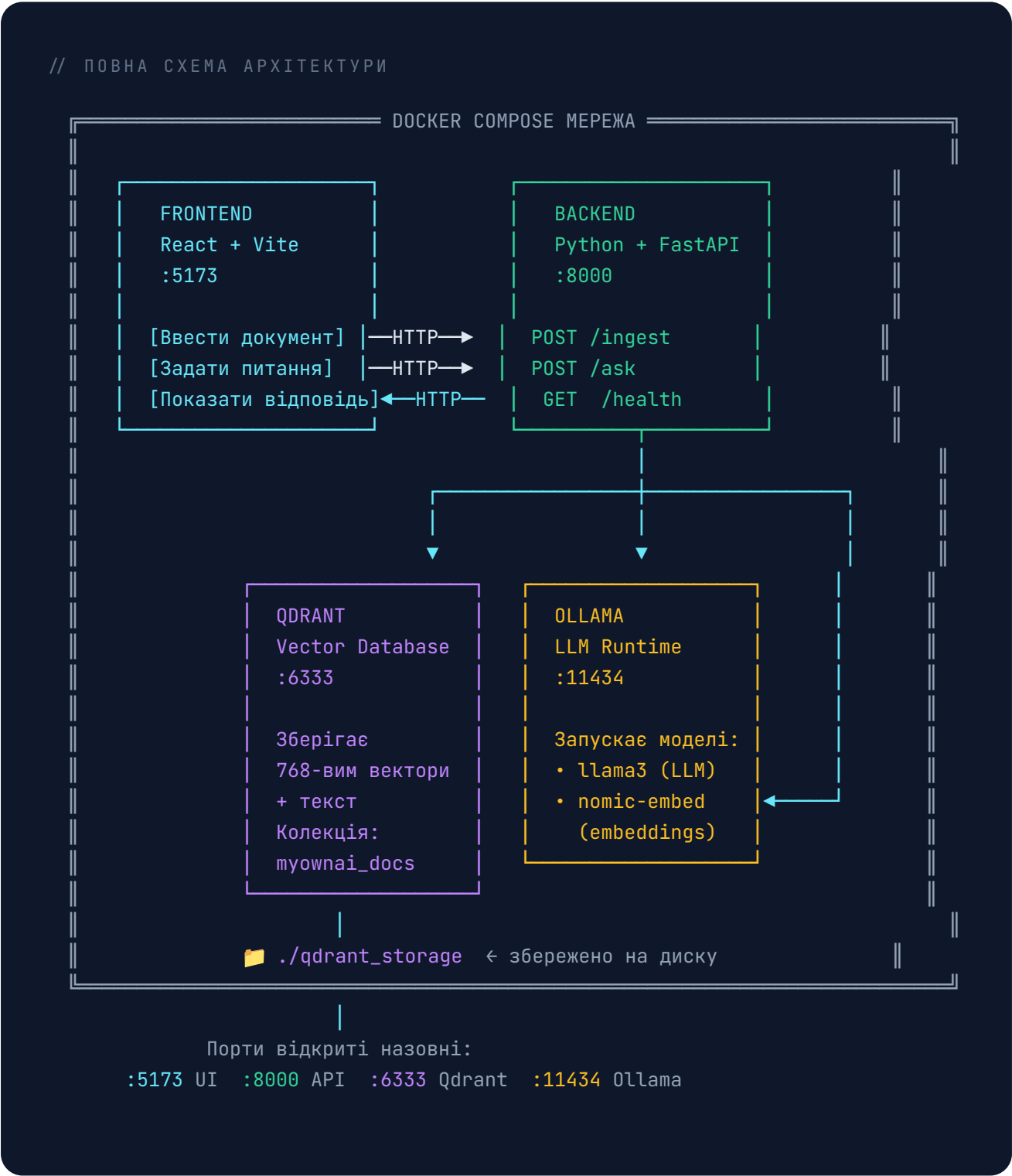
|

"Кіт відпочиває на покрівлі" → `[0.24, -0.85, 0.39, ...]` БЛИЗЬКИЙ ✓

"Фінансовий звіт за 2023 рік" → `[0.91, 0.33, -0.72, ...]` ДАЛЕКИЙ ✗

Загальна архітектура системи

MyOwnAI складається з 4 окремих сервісів, кожен з яких запускається в Docker-контейнері. Вони спілкуються між собою через HTTP і разом утворюють повноцінний AI-застосунок.



Хто від кого залежить?

```
// ЗАЛЕЖНОСТІ (DEPENDS_ON У DOCKER-COMPOSE)
```

```
frontend  —залежить від→ backend  
backend   —залежить від→ qdrant  
backend   —залежить від→ ollama
```

Тобто порядок запуску Docker Compose:

```
qdrant → ollama → backend → frontend
```

Сервіси та технології

Кожен сервіс — це окрема програма в Docker-контейнері зі своєю задачею. Ось пояснення кожного компонента та причини його вибору.



Frontend

REACT + VITE

Веб-інтерфейс застосунку. React — популярна JavaScript-бібліотека для побудови UI. Vite — надшвидкий інструмент розробки, що автоматично оновлює сторінку при зміні коду.

:5173



Backend

PYTHON + FASTAPI

Серцевина логіки. FastAPI — сучасний Python-фреймворк для API. Автоматично генерує документацію, дуже швидкий і простий у написанні.

:8000



Qdrant

VECTOR DATABASE

Спеціалізована база даних для зберігання та пошуку векторних ембедингів. Замість пошуку по ключовим словам — шукає за семантичною схожістю.

:6333



Ollama

LOCAL LLM RUNTIME

Платформа для запуску AI-моделей локально на твоєму комп'ютері. Жодних API-ключів, жодних хмарних сервісів — все працює офлайн.

:11434



LangChain

AI ORCHESTRATION FRAMEWORK

Бібліотека Python, що пов'язує все разом: LLM, векторні бази, ембединги. Без неї довелося б писати сотні рядків клею між сервісами вручну.



Docker Compose

CONTAINER ORCHESTRATION

Дозволяє описати всі 4 сервіси в одному файлі і запустити їх однією командою. Кожен сервіс ізольований, але може спілкуватися з іншими.

Моделі в Ollama

Модель	Тип	Розмір	Навіщо
<code>llama3</code>	LLM (велика мовна модель)	~4.7 GB	Генерує відповіді на питання на основі контексту
<code>nomic-embed-text</code>	Embedding модель	~274 MB	Перетворює текст у 768-вимірні вектори для пошуку



Чому Ollama, а не OpenAI/Claude?

Повна приватність — дані не залишають твій комп'ютер. Безкоштовно без API-ключів. Ти контролюєш модель і можеш змінити її в будь-який момент через змінні середовища.

Змінні середовища (Environment Variables)

Налаштування передаються через env-змінні в `docker-compose.yml` :

DOCKER-COMPOSE.YML — BACKEND ENVIRONMENT

environment:

- OLLAMA_BASE_URL=http://ollama:11434 # URL Ollama всередині Docker-мережі
- QDRANT_URL=http://qdrant:6333 # URL Qdrant всередині Docker-мережі
- COLLECTION_NAME=myownai_docs # Назва колекції в Qdrant
- LLM_MODEL=llama3 # Яку LLM використовувати
- EMBEDDING_MODEL=nomic-embed-text # Яку embedding-модель використовувати
- VECTOR_SIZE=768 # Розмірність векторів



Важливо: ollama vs localhost

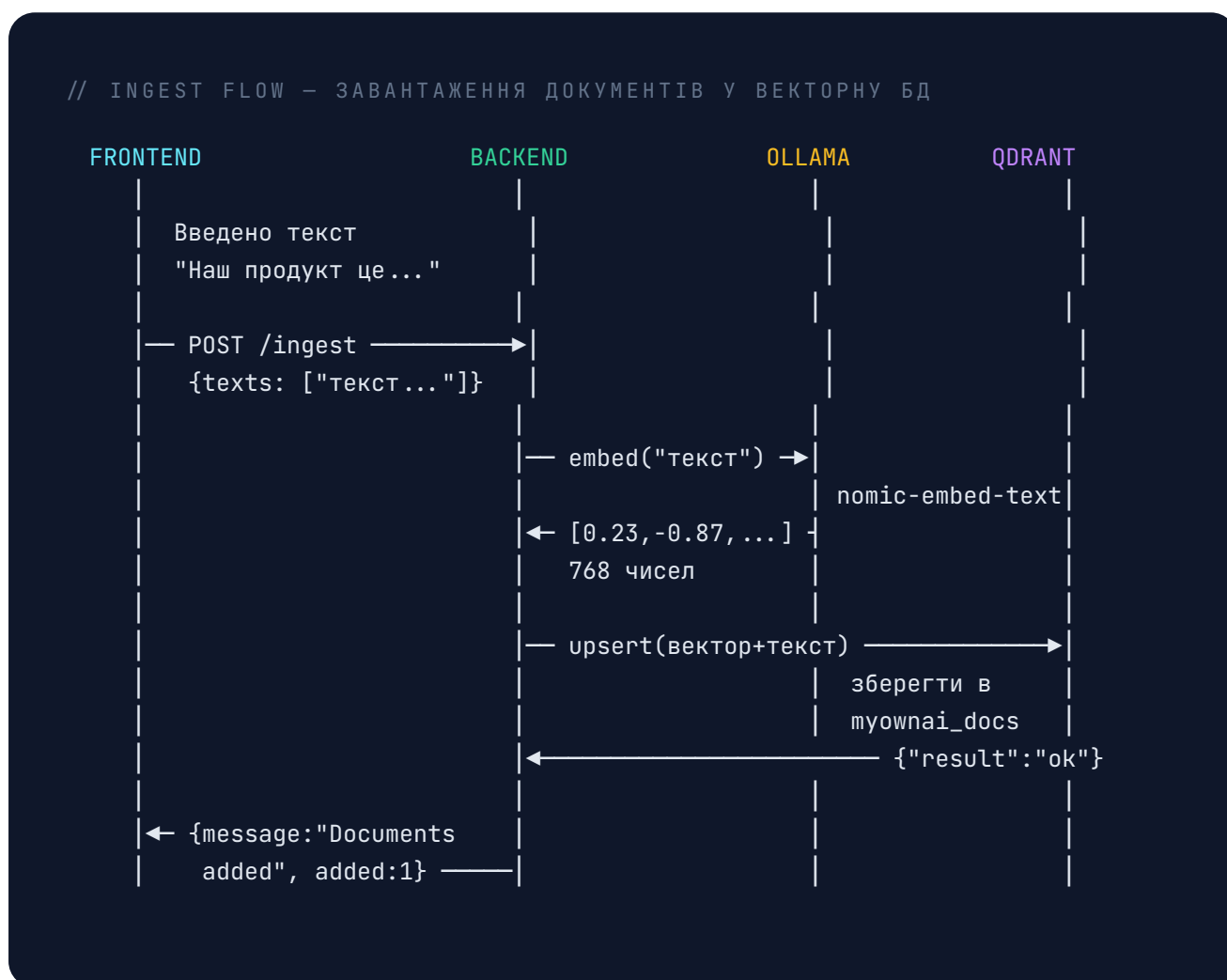
Всередині Docker-мережі сервіси звертаються один до одного по **імені сервісу** (наприклад `http://ollama:11434`), а не по `localhost`. Localhost — це лише для звернень з твого комп'ютера ззовні контейнерів.

Потік даних: як працює запит

В системі є два головних процеси: завантаження документів (Ingest) та отримання відповіді (Ask). Нижче — детальний покроковий розбір кожного.

Процес 1: Завантаження документів (Ingest)

Перш ніж AI зможе відповідати, потрібно "навчити" систему на твоїх документах:



1

Користувач вводить текст у Frontend

Вставляє документ або будь-який текст у поле "Текст для індексації" та натискає "Add to Qdrant".

2

Frontend надсилає POST /ingest

React-додаток робить HTTP-запит до Backend з текстом у тілі запиту у форматі JSON.

3

Backend просить Ollama створити ембединг

LangChain викликає `OllamaEmbeddings.embed_documents()` → Ollama запускає `nomiс-embed-text` і повертає вектор з 768 чисел.

4

Backend зберігає вектор у Qdrant

LangChain викликає `QdrantVectorStore.add_documents()` → Qdrant зберігає пару: вектор + оригінальний текст у колекції `myownai_docs`.

Процес 2: Запит та отримання відповіді (Ask/RAG)

```
// ASK FLOW – RAG ЗАПИТ: ПОШУК + ГЕНЕРАЦІЯ
```

FRONTEND

BACKEND

OLLAMA

QDRANT

Питання: "Що таке
наш продукт?"

— POST /ask —→

— embed(питання)▶

← [0.21, -0.83, ..] nomiс-embed

— similarity_search(вектор, k=3) ▶
пошук 3 найближчих

← [doc1, doc2, doc3]

Формуємо промпт:

"Контекст:

[doc1, doc2, doc3]

Питання: ..."

— chat(промт) →

← "Наш продукт є
..."

llama3 генерує
відповідь

← {answer: "...",
sources: [doc1, doc2 ...]}

1

Питання → вектор

Питання користувача також перетворюється в ембединг через `nomiс-embed-text`.

2

Пошук схожих документів у Qdrant

Backend шукає в Qdrant 3 документи, вектори яких найближчі до вектора питання. Це і є семантичний пошук.

3

Формування промпту з контекстом

Backend будує спеціальний промпт: "Ось контекст: [знайдені документи]. Відповідай ТІЛЬКИ на основі цього контексту. Питання: ..."

4

Ллама 3 генерує відповідь

Ollama запускає `llama3` з цим промптом і повертає текстову відповідь, сформовану з урахуванням знайденого контексту.

5

Відповідь + джерела → Frontend

Backend повертає і відповідь, і список документів-джерел, щоб користувач міг перевірити, звідки взята інформація.

Backend — FastAPI

Backend — мозок системи. Написаний на Python з FastAPI. Знаходиться в `backend/app/main.py`. Ось що там відбувається при запуску та на кожен запит.

Що відбувається при старті сервера

// STARTUP SEQUENCE — ПОСЛІДОВНІСТЬ ІНІЦІАЛІЗАЦІЇ

uvicorn запускає FastAPI

↓

1. Зчитуємо змінні середовища
OLLAMA_BASE_URL, QDRANT_URL, LLM_MODEL ...

↓

2. `wait_for_qdrant()` — чекаємо на Qdrant
Пробуємо підключитись 20 разів з паузою 1с
Якщо не підключилось — завершуємо роботу

↓

3. `ensure_collection_exists()`
Перевіряємо чи є колекція "myownai_docs"
Якщо немає — створюємо з векторами розміром 768

↓

4. `QdrantVectorStore.from_existing_collection()`
Підключаємось до колекції через LangChain
↳ Тут і сталась помилка з `pojie-embed-text!`
Перевіряє розмір вектора — для цього викликає `embed("dummy_text")`

↓

5. `ChatOllama(model="llama3", temperature=0.2)`
Ініціалізуємо LLM клієнт для llama3

↓

Сервер готовий — Application startup complete ✓

API Ендпоінти

GET

/health

Перевірка стану сервера

Повертає статус сервера та конфігурацію. Використовується для моніторингу і healthcheck-скриптів.

ВІДПОВІДЬ JSON

```
{
  "status": "ok",
  "stack": ["React", "FastAPI", "Ollama", "Llama 3", "LangChain", "Qdrant", "Docker"],
  "qdrant_collection": "myownai_docs",
  "embedding_model": "nomic-embed-text",
  "vector_size": 768
}
```

POST

/ingest

Завантаження документів

Запит (Request Body)

JSON

```
{
  "texts": [
    "Перший документ...",
    "Другий документ..."
  ]
}
```

Відповідь (Response)

JSON

```
{
  "message": "Documents added",
  "added": 2
}
```

Запит

```
JSON

{
  "question": "Що таке наш продукт?"
}
```

Відповідь

```
JSON

{
  "answer": "Ваш продукт це...",
  "sources": [
    "Текст документа 1...",
    "Текст документа 2...",
    "Текст документа 3..."
  ]
}
```

 Swagger UI — вбудована документація API

FastAPI автоматично генерує інтерактивну документацію. Відкрий у браузері: <http://localhost:8000/docs> — там можна тестувати всі ендпоінти без Frontend.

Frontend — React

Інтерфейс написаний на React і складається фактично з одного файлу `frontend/src/main.jsx`. Простий і ефективний.

Структура UI

```
// МАКЕТ ІНТЕРФЕЙСУ
```



MyOwnAI

Local RAG AI powered by Ollama

— СЕКЦІЯ 1: Додати документ —

textarea: введи текст для індексації...

[Add to Qdrant] ← POST /ingest

— СЕКЦІЯ 2: Задати питання —

input: Ваше питання...

[Ask AI] ← POST /ask

— СЕКЦІЯ 3: Відповідь —

Відповідь AI:

"Ваш продукт дозволяє..."

Знайдені джерела (контекст):

- "Документ 1: Наш продукт є..."
- "Документ 2: Продукт призначений для..."

Стан (State) React-компонента

JAVASCRIPT — USESTATE ГАЧКИ

```
const [text, setText]      = useState('')    // текст документу для завантаження
const [question, setQuestion] = useState('')  // питання користувача
const [answer, setAnswer]   = useState('')    // відповідь від AI
const [sources, setSources] = useState([])    // знайдені документи-джерела
const [loading, setLoading] = useState(false) // прапорець завантаження
```

Як React взаємодіє з Backend

```
// REACT → BACKEND ВЗАЄМОДІЯ (FETCH API)

// Кнопка "Add to Qdrant" — завантаження документу
async handleIngest() {
  setLoading(true)
  await fetch('http://localhost:8000/ingest', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ texts: [text] })
  })
  setLoading(false)
}

// Кнопка "Ask AI" — RAG запит
async handleAsk() {
  setLoading(true)
  const res = await fetch('http://localhost:8000/ask', {
    method: 'POST',
    body: JSON.stringify({ question: question })
  })
  const data = await res.json()
  setAnswer(data.answer)    // відповідь AI
  setSources(data.sources)  // список джерел
  setLoading(false)
}
```

Структура файлів Frontend

Файл	Призначення
<code>src/main.jsx</code>	Головний React-компонент — весь UI і логіка
<code>src/style.css</code>	Стили: темна тема, кольори, відступи
<code>index.html</code>	HTML-шаблон з <code>div#root</code> куди монтується React
<code>package.json</code>	Залежності: react, react-dom, vite
<code>Dockerfile</code>	node:20-alpine, запускає Vite dev-сервер

Запуск та розгортання

Проект розгортається через Docker Compose. Є декілька способів — від швидкого до повного з нуля.

Перший запуск (покроково)

1

Запустити всі сервіси

TERMINAL

```
make deploy          # або: ./deploy.sh
# також можна:
make up               # docker compose up -d --build
```

2

Завантажити AI-моделі (один раз!)

TERMINAL

```
make pull-models
# Завантажує ~5 GB:
# • nomic-embed-text (274 MB) — для ембедингів
# • llama3            (~4.7 GB) — для генерації відповідей
```

3

Перевірити що все працює

TERMINAL

```
make health          # перевіряє всі сервіси
```

URL

<http://localhost:5173> ← Головний UI
<http://localhost:8000/docs> ← Swagger API документація
<http://localhost:6333/dashboard> ← Qdrant веб-панель

Всі команди Makefile

Команда	Що робить
<code>make deploy</code>	Повне розгортання через <code>deploy.sh</code> (рекомендовано)
<code>make deploy-clean</code>	Розгортання без кешу Docker (якщо щось зламалось)
<code>make up</code>	Запустити сервіси з білдом образів
<code>make down</code>	Зупинити і видалити контейнери
<code>make pull-models</code>	Завантажити llama3 і nomic-embed-text в Ollama
<code>make backend-logs</code>	Показати логи Backend в реальному часі
<code>make frontend-logs</code>	Показати логи Frontend в реальному часі
<code>make logs</code>	Показати всі логи
<code>make health</code>	Перевірити стан всіх сервісів
<code>make reset</code>	Очистити базу Qdrant (видалити всі документи)

Структура файлів проекту

```
// ДЕРЕВО ФАЙЛІВ ПРОЕКТУ
```

```
My AI test Project/
```

```
|— docker-compose.yml  ← опис всіх 4 сервісів
|— Makefile            ← зручні команди (make deploy, make logs...)
|— deploy.sh           ← скрипт повного розгортання
|— backend/
|   |— Dockerfile      ← python:3.11-slim, uvicorn
|   |— requirements.txt ← залежності Python
|   |— app/
|       |— main.py      ← ВСЯ логіка бекенду (175 рядків)
|— frontend/
|   |— Dockerfile      ← node:20-alpine, vite dev server
|   |— package.json    ← react, react-dom, vite
|   |— index.html      ← точка входу
|   |— src/
|       |— main.jsx     ← весь UI і логіка фронтенду
|       |— style.css    ← темна тема
|— qdrant_storage/     ← тут Qdrant зберігає вектори (на диску)
|— scripts/
|   |— logs.sh         ← перегляд логів
|   |— healthcheck.sh  ← перевірка стану сервісів
|   |— reset.sh        ← очищення бази
```

Типові проблеми та рішення

Найчастіші помилки та як їх виправляти — на основі реальних проблем у цьому проекті.

Помилка: model "nomic-embed-text" not found

 **Ця помилка трапилась у вашому проекті!**

Backend не може запуститись, бо Ollama не має завантаженої моделі ембедингів.

ПРИЧИНА + РІШЕННЯ

```
# Ollama запущений але модель не завантажена
# Виправлення:
make pull-models
# або вручну:
docker exec myownai-ollama ollama pull nomic-embed-text
docker exec myownai-ollama ollama pull llama3
# Після цього перезапустити backend:
docker restart myownai-backend
```

Помилка: Backend не може підключитись до Qdrant

ПРИЧИНА + РІШЕННЯ

```
# Qdrant ще не готовий коли Backend стартує
# Backend сам чекає 20 спроб по 1 секундi (wait_for_qdrant)
# Якщо все одно не працює – перевір статус:
docker ps
# Якщо qdrant не running:
docker start myownai-qdrant
docker restart myownai-backend
```

Помилка: CORS error у браузері

ПРИЧИНА + РІШЕННЯ

```
# Frontend на localhost:5173 не може звертатись до Backend
# CORS вже налаштований для localhost:5173 в main.py
# Якщо ти змінив порт Frontend — додай його в CORS:
# backend/app/main.py:
allow_origins=["http://localhost:5173", "http://localhost:ВАШ_ПОРТ"]
```

Backend запускається але /ask не відповідає

РІШЕННЯ

```
# 1. Перевір чи llama3 завантажений:
docker exec myownai-ollama ollama list

# 2. Перевір логи backend:
make backend-logs

# 3. Якщо llama3 не завантажений:
docker exec myownai-ollama ollama pull llama3
```

Хочу скинути всі документи в Qdrant

РІШЕННЯ

```
make reset    # видалить qdrant_storage і пересоздасть порожній
```

Корисні команди для діагностики

```
// ДІАГНОСТИЧНІ КОМАНДИ

# Статус всіх контейнерів
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"

# Логи конкретного сервісу в реальному часі
docker logs myownai-backend -f --tail 50
docker logs myownai-ollama -f --tail 20

# Список завантажених моделей Ollama
docker exec myownai-ollama ollama list

# Колекції в Qdrant (через браузер або curl)
curl http://localhost:6333/collections

# Скільки документів у колекції
curl http://localhost:6333/collections/myownai_docs

# Перезапустити один сервіс
docker restart myownai-backend
```

Зміна LLM моделі

Хочеш використати іншу модель (наприклад `mistral` або `gemma3`)?

```
DOCKER-COMPOSE.YML → BACKEND → ENVIRONMENT

- LLM_MODEL=mistral    # змінити тут
# Потім завантажити нову модель:
docker exec myownai-ollama ollama pull mistral
# Перезапустити backend:
docker restart myownai-backend
```



Все працює коректно, якщо:

- `docker ps` показує 4 контейнери зі статусом **Up**
- `ollama list` показує **nomic-embed-text** і **llama3**
- `http://localhost:8000/health` повертає **"status": "ok"**
- Frontend відкривається на `http://localhost:5173`